for answering queries. This can be a static or fixed property (e.g., one year) or something that you can configure (potentially, along with downsampling strategies in metrics backends, like CNCF Thanos already offers).

- *Query costs*—Using the signals to answer your o11y questions means you need to be able to look them up, scan them, and query them; these costs can be expressed in samples processed or can be more volume oriented (e.g., GB scanned). Understanding where and how query occurs (besides the maybe obvious ad hoc query you, as a human, perform interactively) is crucial.

  For example, think of the case in which you define an alert or alarm (compare chapter 7), and then a condition is evaluated in regular time periods, causing one or more queries to be executed. Again, at scale, this can sum up, and I have seen surprises (not the good kind) from customers not aware of this.

In addition to these variable costs, there can be flat costs, such as per-seat license costs; however, those are fairly predictable (in contrast to the variable costs, such as ingestions or query) and more commonly found with frontends or all-in-one solutions. One more consideration in the context of the costs of a backend—beyond the actual task at hand, like troubleshooting or performance optimization— is that there may be regulatory requirements to keep certain types of signals (usually logs) around for several years.

> **NOTE** Regulatory requirements around the storage and protection of telemetry signals is especially pertinent in regulated verticals, such as financial industries or health care. You may need to prove this capability in the context of achieving a certification, such as the case of an app that handles credit cards usually requiring Payment Card Industry Data Security Standard (PCI DSS) compliance or privacy requirements in the context of System and Organization Controls (SOC).

Here is one quick word on buy versus build in the context of backends: unless you are an observability vendor (or cloud provider), please consider not building your own backend from scratch. It can take many years to build something reliable and scalable, and if you have *N* innovation tokens or engineering cycles to spend, they are usually better invested in your core business and what sets you apart from the competition, rather than building the next time series data store.

I do not mean to discourage you from building an internal observability platform based on, for example, integrating various open source backends, with an open source frontend as a single pane of glass. However, you should be aware of the total cost of ownership (TCO), so be prepared to take the costs of your engineering team, support, training, and security patching (to name a few common ones) into account when you make a decision. Let's move on to the topic of open source and open standards next.